

Comprehensive System Documentation: Rocket-GPS-INS-Fusion



Team: Infinity Explorers | **Course:** SNS312 Guidance and Navigation Systems

1. Executive Summary & Flight Profile Objective

This project designs, models, and evaluates an integrated Global Positioning System (GPS) and Inertial Navigation System (INS) for a vertically ascending rocket. The core objective is to track the trajectory across four distinct flight phases: **Boost** (high acceleration), **Coast** (deceleration under gravity), **Apogee** (zero vertical velocity), and **Descent/Recovery**.

By estimating the rocket's altitude and vertical velocity using a discrete 1D Kalman filter, the system resolves the inherent flaws of both sensors: the rapid integration drift of the INS and the low update rate/noise of the GPS.

Hardware-in-the-Loop (HIL) Implementation: Moving beyond theoretical simulation, this project fuses real-world hardware telemetry—a high-frequency, drifting MPU6050 and a low-frequency, noisy NEO-6M GPS—demonstrating real-time navigation capabilities bridging theoretical models with physical aerospace applications.

2. Hardware Architecture & Embedded Layer (Omar's Domain)

The embedded tier (`daq_system.ino`) is deployed on an Arduino-compatible microcontroller. It is designed around strict deterministic timing, which is an absolute necessity for minimizing numerical integration errors.

2.1 Sensor Polling Strategy & I2C Optimization

Microcontrollers operate sequentially. If the processor blocks execution to wait for a 1 Hz GPS update, the INS integration loop is destroyed. The embedded code isolates the GPS and IMU routines:

- **MPU6050 (INS):** Driven by a non-blocking hardware timer (`millis() - lastTime >= 10`), ensuring exactly 100 Hz polling (10 ms Δt). To achieve this without bottlenecking, the I2C bus clock is explicitly set to Fast Mode (400 kHz) via `Wire.setClock(400000)` .
- **NEO-6M / TinyGPS++:** The SoftwareSerial buffer (operating at 9600 baud) is continuously drained via a `while(ss.available() > 0)` loop. The NMEA sentences (GGA, RMC) are parsed asynchronously without interrupting the 100 Hz IMU timer.

2.2 Binary Packetization & Serial Transport

To support high-speed data transfer at a 500,000 baud rate to the ground station, ASCII text transmission (`Serial.print()`) is abandoned in favor of a tightly packed C++ `struct` . This reduces byte overhead and guarantees atomic data transmission.

```

struct __attribute__((packed)) TelemetryPacket {
    uint16_t header = 0xBBAA; // Sync Word
    uint32_t timestamp;        // Milliseconds since boot
    float accel_x, accel_y, accel_z;
    float gps_alt, gps_vel;
    uint8_t gps_updated;       // Boolean flag for valid 3D Fix
};

```

This 27-byte packet ensures the Python backend can safely synchronize with the data stream even under high serial load without causing fragmentation. A standard UART frame uses 10 bits (8 data, 1 start, 1 stop). At 500,000 baud, transmitting 27 bytes takes approximately 0.54 ms, safely within our 10 ms budget.

3. Software Architecture & Python Pipeline (Mostafa's Domain)

The system integrator utilizes a high-performance, multithreaded **Producer-Consumer architecture** mapped primarily in `main.py`. This ensures mathematical integration loops are not bottlenecked by the Global Interpreter Lock (GIL) or UI rendering calls.

- **SerialManager (Thread / Producer):** Listens to the COM port using `pyserial`. It scans the byte stream byte-by-byte until it aligns with the `0xBBAA` sync word. It then reads the next 25 bytes, unpacks the data using `struct.unpack('<HIffffB')`, populates a `SensorData` dataclass (`data_types.py`), and pushes it to a thread-safe `queue.Queue`.
- **DataProcessor (Thread / Consumer):** Pulls from the queue, computes precise time deltas (Δt from hardware timestamps), executes the Kinematics class, and triggers the Kalman Filter. It emits calculated states to the main thread via thread-safe Qt Signals (`pyqtSignal`).
- **Visualizer (Main Thread):** Runs the PyQtGraph GUI (`visualizer.py`). It receives signals from the DataProcessor and appends them to NumPy rolling buffers for hardware-accelerated rendering.

4. Mathematical Modeling & Kinematics (Mariam's Domain)

Defined in `kinematics.py`.

4.1 Initialization and Gravity Calibration

Before launch, the system enters a calibration phase. The first 100 samples (1 second) of stationary data are averaged to calculate the local gravity vector and initial baseline noise.

$$a_{z_corrected} = a_z - g_{local}$$

4.2 Trapezoidal Integration

To mitigate the aggressive compound error of standard Euler integration, the system utilizes Trapezoidal Integration. By averaging the previous and current acceleration values, the

algorithm constructs sloped lines rather than flat rectangles between discrete time steps:

$$v_k = v_{k-1} + \frac{1}{2}(a_k + a_{k-1})\Delta t$$

$$p_k = p_{k-1} + \frac{1}{2}(v_k + v_{k-1})\Delta t$$

4.3 The Dead Reckoning Trap

Despite the accuracy of trapezoidal math, double-integrating inherent hardware bias transforms a constant linear error into a quadratic parabolic drift. If the accelerometer has a constant bias ϵ :

$$\text{Position Error}(t) = \iint \epsilon dt^2 = \frac{1}{2}\epsilon t^2$$

This demonstrates mathematically why INS-only tracking fails rapidly over long durations, making sensor fusion mandatory.

5. Sensor Fusion / Discrete Kalman Filter (Tasneem's Domain)

Defined in `kalman_filter.py`. The filter dynamically tracks three variables in a 1D vertical model: Altitude (p), Velocity (v), and Sensor Bias (b).

$$X = \begin{bmatrix} p \\ v \\ b \end{bmatrix}$$

5.1 Initialization and Tuning Parameters

The filter requires accurate initialization of noise covariance matrices:

- **Process Noise Covariance (Q):** Defines trust in the IMU. Tuned using Allan Variance analysis of the MPU6050.

$$Q = \begin{bmatrix} \sigma_p^2 & 0 & 0 \\ 0 & \sigma_v^2 & 0 \\ 0 & 0 & \sigma_b^2 \end{bmatrix}$$

- **Measurement Noise Covariance (R):** Defines trust in the GPS. For the NEO-6M, altitude variance is high. $R = [15.0]$.
- **Initial Error Covariance (P_0):** Set high to allow the filter to aggressively converge on the first few GPS measurements.

5.2 The Prediction Step (100 Hz)

The State Transition Matrix (F) advances the state. The matrix mathematically subtracts the estimated bias (b) from the raw acceleration, correcting both velocity and position before integration.

$$F = \begin{bmatrix} 1 & \Delta t & -0.5\Delta t^2 \\ 0 & 1 & -\Delta t \\ 0 & 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 0.5\Delta t^2 \\ \Delta t \\ 0 \end{bmatrix}$$

$$X_{k|k-1} = F \cdot X_{k-1|k-1} + B \cdot a_{z_corrected}$$

$$P_{k|k-1} = F \cdot P_{k-1|k-1} \cdot F^T + Q$$

5.3 The Update Step (1 Hz)

When `gps_updated` is true (3D satellite fix), the Measurement Matrix $H = [1, 0, 0]$ isolates altitude. The Kalman Gain (K) weighs the INS uncertainty against GPS measurement noise. To maintain numerical stability and ensure the P matrix remains positive semi-definite, the system uses the **Joseph form** for the covariance update.

$$y = z_{gps} - H \cdot X_{k|k-1} \quad (\text{Innovation})$$

$$S = H \cdot P_{k|k-1} \cdot H^T + R \quad (\text{Innovation Covariance})$$

$$K = P_{k|k-1} \cdot H^T \cdot S^{-1} \quad (\text{Kalman Gain})$$

$$X_{k|k} = X_{k|k-1} + K \cdot y \quad (\text{State Update})$$

$$P_{k|k} = (I - KH) \cdot P_{k|k-1} \cdot (I - KH)^T + K R K^T \quad (\text{Joseph Form Update})$$

6. Visualization & Data Logging (Alaa's Domain)

The `visualizer.py` script presents a 4-pane PyQtGraph dashboard designed for real-time telemetry assessment without blocking the main CPU thread:

1. **Raw GPS Altitude:** Blocky, low-update-rate trajectory heavily influenced by satellite noise variance.
2. **Raw INS Kinematics:** Perfectly smooth trajectory that aggressively diverges due to uncorrected bias integration.
3. **Kalman Filter (Smoothed):** Isolated output showing rejection of high-frequency noise while correcting the baseline.
4. **Sensor Fusion Overview:** Overlays all datasets, visually proving the algorithm's efficacy.

Simultaneously, `main.py` streams the raw and filtered data into a timestamped CSV file (`/logs/flight_log_YYYYMMDD_HHMMSS.csv`). This enables rigorous post-flight quantitative analysis via MATLAB or Pandas.

7. Discussion and Analysis

Addressing Section 5 of the project rubric, the hardware-in-the-loop implementation revealed several key practical dynamics:

7.1 Sensor Noise, Bias, and Sampling Rates

The discrepancy in sampling rates is the primary challenge in aerospace avionics. The software architecture elegantly handles this: the Kalman Filter loops the Prediction step 100 times per second (providing smooth flight control data), but only executes the Update step once per second when the GPS triggers.

7.2 Simulation vs. Hardware Results

In pure simulation, noise is modeled purely as Gaussian white noise. In hardware, thermal expansion on the PCB and mechanical vibrations from the rocket motor inject non-Gaussian, low-frequency biases. This proves that tracking the accelerometer bias (the b state variable) is not optional—without it, the filter cannot correct INS drift fast enough between 1 Hz GPS updates.

7.3 Limitations and Future Scope (6-DOF)

The current 1D model assumes perfectly vertical flight. In a real rocket, pitch and yaw rotations cause the Z-axis accelerometer to bleed gravity into the horizontal X/Y axes. Future iterations of this project will implement an **Extended Kalman Filter (EKF)** utilizing quaternions to track a full 6 Degrees of Freedom (6-DOF) attitude. This will allow the system to rotate the local body-frame acceleration vectors into the global navigation frame before subtracting gravity.